Multithreading

==========

Multithreading in Java is a core platform feature that allows the **concurrent execution of two or more parts of a program** to maximize CPU utilization. Each independent path of execution within the program is called a **thread**, which behaves like a lightweight sub-process sharing the same memory space.

Why Use Multithreading?

- **Better CPU usage**: Executes multiple tasks simultaneously.
- **Improved responsiveness**: Prevents background tasks from freezing the UI.
- **Resource sharing**: Threads utilize the same memory space, reducing context-switch overhead.

How to Create Threads in Java

Java natively supports thread creation through two primary mechanisms:

1. By Extending the `Thread` Class

You create a custom class that inherits from `java.lang.Thread` and override its `run()` method.

java

```
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread thread = new MyThread();
        thread.start(); // Spawns a new thread and calls run()
    }
}
```

2. By Implementing the `Runnable` Interface

This is the **preferred approach** because Java does not support multiple class inheritance. Implementing an interface leaves your class free to extend another base class.

java

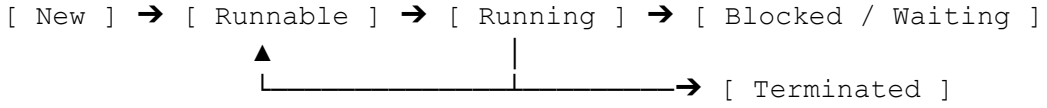
```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread thread = new Thread(new MyRunnable());
        thread.start();
    }
}
```

Note: Always call `start()` to launch a new thread. Calling `run()` directly will execute the code on the current thread without creating a new one.

The Thread Lifecycle

A thread transitions through several distinct states managed by the JVM and OS scheduler:



1. **New**: The thread instance is created but `start()` has not been called.
2. **Runnable**: `start()` is invoked, and the thread is eligible to run but waiting for the CPU scheduler.
3. **Running**: The thread scheduler has allocated a CPU slot; the `run()` code executes.
4. **Blocked / Waiting**: The thread is temporarily inactive (e.g., waiting for an I/O operation, sleeping via `Thread.sleep()`, or waiting for a resource lock).
5. **Terminated**: The `run()` method completes its execution or gets aborted.

Key Concurrency Concepts

- **Synchronization**: When multiple threads access shared mutable data, data inconsistency (race conditions) can occur. Java uses the `synchronized` keyword to lock code blocks or methods so only one thread can execute them at a time.
- **Inter-Thread Communication**: Threads can coordinate using `wait()`, `notify()`, and `notifyAll()` methods to avoid CPU-wasting loops (polling).
- **Deadlock**: A problematic situation where two or more threads are permanently blocked, each waiting for a resource held by the other.

For complex applications, managing raw `Thread` objects can become inefficient. Developers typically use the modern Java Executor Framework via the `java.util.concurrent` package to efficiently manage thread pools.

Various ways to create Threads in java.

In Java, there are **four / five main ways** to create and manage threads. While the first two are the classic foundational methods, the last two are modern approaches used in professional production environments.one is using java 8.

---

1. By Extending the `Thread` Class

You create a new class that inherits from `java.lang.Thread` and override its `run()` method.

- **Pros**: Simple to write for basic tasks.
- **Cons**: Java only allows extending one class. You cannot inherit from any other class.

java

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running!");
    }
}

// How to start it:
MyThread t = new MyThread();
t.start();
```

---

2. By Implementing the `Runnable` Interface

You implement the `Runnable` interface and pass its instance to a `Thread` object. This is the **standard traditional way**.

- **Pros**: Frees up your class to extend a different parent class if needed.
- **Cons**: The `run()` method cannot return a result or throw checked exceptions.

java

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running!");
    }
}

// How to start it:
Thread t = new Thread(new MyRunnable());
t.start();\
```

```
(JAVA 8)
```

(Tip: In modern Java, you can shorten this using a Lambda expression: `Thread t = new Thread(() -> System.out.println("Running!"));`)

---

3. By Implementing the `Callable` Interface (with `Future`)

Introduced to fix the limitations of `Runnable`. It must be executed using an `ExecutorService`.

- **Pros**: Can **return a data value** after finishing and can throw checked exceptions.
- **Cons**: Slightly more complex boilerplate code to set up.

java

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

class MyCallable implements Callable<String> {
    public String call() throws Exception {
        return "Task Completed!";
    }
}
```

```
// How to start it:
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<String> future = executor.submit(new MyCallable());

// Retrieve the result later:
String result = future.get();
executor.shutdown();
```

---

4. By Using the Thread Pool / Executor Framework

Instead of manually creating `Thread` objects every time, you use a pre-built pool of managed threads.

- **Pros**: Highly efficient for large applications because it reuses existing threads instead of wasting CPU creating new ones.
- **Cons**: Requires managing the lifecycle of the pool (shutting it down properly).

java

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

// Create a pool of 3 reusable threads
ExecutorService pool = Executors.newFixedThreadPool(3);

// Submit multiple tasks to the pool
pool.execute(() -> System.out.println("Task 1 running"));
pool.execute(() -> System.out.println("Task 2 running"));

// Turn off the pool when done
pool.shutdown();
```

Summary Comparison

Method	Can Return Value?	Can Throw Checked Exception?	Reuses Threads?
1. Thread Class	❌ No	❌ No	❌ No
2. Runnable	❌ No	❌ No	❌ No
3. Callable	Yes	Yes	❌ No (unless pooled)
4. Executor Framework	Yes (with Callable)	Yes (with Callable)	**Yes**

Is callable and executer both are same or different?

They are **different** but they **work together as a team**.

Think of it like a restaurant:

- **Callable** is the **Order** (the food item you want made).
- **Executor** is the **Chef** (the worker who actually cooks it).

Here is the exact breakdown of how they differ and how they connect:

The Core Difference

- **Callable is a Task:** It is just an interface that defines **what** code needs to run. It holds the work instructions, can return a result, and can throw errors. It cannot run on its own.
- **Executor is a Manager:** It is a service that decides **how and when** to run tasks. It manages a pool of background threads to execute the `Callable` tasks you give it.

Comparison at a Glance

Feature	Callable	Executor (ExecutorService)
What is it?	A **Task** definition	A **Thread Manager**
Does it create threads?	❌ No	**Yes** (manages a pool of threads)
Main Job	Holds the code to execute	Runs the code and allocates CPU
Key Method	`call()`	`submit()` or `execute()`

How They Work Together

You create a `Callable` task, and then you hand it over to an `Executor` to run it. The Executor gives you back a `Future` object, which acts like a claim ticket to collect your result later.

java

```
// 1. Define the task (The Order)
Callable<Integer> calculateTask = () -> {
    return 5 + 5;
};

// 2. Create the executor (The Chef)
ExecutorService executor = Executors.newSingleThreadExecutor();

// 3. Hand the task to the executor (The Chef cooks the order)
Future<Integer> ticket = executor.submit(calculateTask);

// 4. Get your result using the ticket
Integer result = ticket.get(); // Returns 10
```

3rd ways vs 4th ways :

3. By Implementing the Callable Interface (with Future)

4. By Using the Thread Pool / Executor Framework what and why is this?

To understand **what** these two modern approaches are and **why** they were created, it helps to understand the problems they solved. Traditional approaches (like `Thread` and `Runnable`) were missing crucial features needed for big, professional applications.

Here is the plain-English breakdown of what they are and why they exist:

Method 3: Implementing the Callable Interface (with Future)

What is it?

`Callable` is a modern replacement for `Runnable`. While a `Runnable` just runs code and returns nothing (`void`), a `Callable` executes code and **returns a calculation result** back to your main program. [1, 2, 3]

Because the thread runs in the background, you cannot get the result instantly. Java handles this by immediately handing you a **`Future` object**. The `Future` object acts like a fast-food receipt or a claim ticket—when the background thread finishes its work, you hand over the ticket (`future.get()`) to collect your actual data. [1, 2]

Why do we need it?

- **Runnable cannot return data:** With `Runnable`, if a background thread calculates a number or fetches a user profile from the internet, there is no built-in way to send that data back to the main thread. Developers had to write complex, messy code with shared variables just to move data out. `Callable` fixes this by safely returning data.

- **Runnable cannot handle checked exceptions:** If your code inside a `Runnable` throws an error (like an internet connection failure), you are forced to catch it right inside the thread using a `try-catch` block. `Callable` lets you throw the error upward so your main application can handle it gracefully.
- **Task Control:** The `Future` ticket lets you track the thread's status. You can check if it is finished (`isDone()`), stop it early (`cancel()`), or set a maximum time limit to wait so your app never freezes indefinitely.

Method 4: Using the Thread Pool / Executor Framework

What is it?

Instead of manually typing `new Thread()` every time you need a background task, you create an `ExecutorService` (a **Thread Pool manager**). You set up a fixed number of threads (for example, 5 threads) that sit idle, waiting for work. When you have a task, you simply throw it into the pool. An idle thread grabs the task, completes it, and then goes back into the pool to wait for the next task.

Why do we need it?

- **Creating threads is expensive:** In Java, creating a physical thread requires a lot of memory and CPU cycles from your computer's operating system. If your app handles 10,000 users and you create a new thread for every single user, your computer will quickly crash or run out of memory. Thread pools solve this by keeping a small, smart group of permanent threads alive and recycling them over and over.
- **Automatic Queueing:** If all the threads in your pool are currently busy working, the `Executor` automatically holds your new tasks in an orderly waiting line (a queue) until a thread becomes free. You do not have to write any code to manage the waiting list.
- **Separation of Concerns:** It separates **what** the task does from **how** it is run. Your code only has to focus on defining the task; the executor framework handles the complicated logistics of scheduling, CPU usage, and thread lifecycles.

Simple Real-World Analogy

Imagine a busy **Package Delivery Hub**:

- **Traditional Threading:** Every single time a customer brings in a package, the hub builds a brand new delivery truck from scratch, hires a new driver, sends them out, and then destroys the truck when the delivery is done. (Highly inefficient!)
- **The Executor Framework (Thread Pool):** The hub buys a fixed fleet of 10 permanent trucks and drivers. They sit in the parking lot waiting. As packages arrive, drivers grab them, deliver them, and return to the lot to wait for the next package.
- **Callable & Future:** When a driver takes a package, they hand the customer a tracking number (`Future`). The customer uses that tracking number to check the delivery status or see the final delivery receipt (`Callable` result).